# Dentro al microcontrollore

Lezione 1 (continua) — 3MAK, SST-MEC-25c — Architetture e circuito di clock

17/09/2026

## 1 Microprocessore o microcontrollore? La differenza

Dentro ogni sistema di elaborazione elettronico ci sono dei dispositivi che, in base a una configurazione di bit, eseguono somme, confronti, decisioni: sono fondamentalmente **porte logiche** combinate tra loro. La sequenza di queste operazioni è il nostro programma.

**Definizione**

Un **microprocessore** è un dispositivo che fa ***solo* calcoli** (la CPU, Central Processing Unit): da solo **non può fare nulla**, perché non ha nessuna memoria propria. Per funzionare deve essere collegato ad almeno **quattro componenti esterni** distinti: una RAM (per i dati), una **memoria per il programma**, un dispositivo di **input/output** (per collegare LED, pulsanti, ecc.), e altro hardware di supporto.
Un **microcontrollore** integra tutto questo – **CPU**, **memoria dati**, **memoria programma**, **periferiche** di input/output – **in un unico chip**. Bastano **pochi componenti esterni** per farlo funzionare.

**Esempio**

Un **processore** Intel o AMD (come quello dentro il PC che usiamo) preso da solo, senza RAM, scheda video, interfacce, **non sa fare nulla**. Un microcontrollore come quello montato su Arduino, invece, con **pochissimi componenti** intorno è già pronto a funzionare.

### 1.1 Sistemi general purpose vs sistemi embedded

**Definizione**

Un **microprocessore** viene tipicamente usato in sistemi **general purpose** (per **usi generici**): lo stesso PC può servire per giocare, per lavorare, per navigare in internet – decide l'utente a cosa usarlo.
Un **microcontrollore** viene invece usato in sistemi **embedded** ("immersi", dedicati): un dispositivo progettato per fare ***una sola cosa***. La scheda elettronica di una lavatrice fa funzionare **solo quella lavatrice**.

**Esempio**

**Non ha senso** vendere ad un utente un "computer" pensato solo per far girare una **lavatrice**, né vendere la scheda di controllo della lavatrice per farci altro (es. giocare): sono due categorie di dispositivi con **scopi opposti**. Un PC general purpose **costa molto di più** (es. 1.500€) di una scheda embedded dedicata (es. 20€) – perché deve poter fare ***tutto***, non solo una cosa.

# 2 Due modi di organizzare la memoria: von Neumann e Harvard

**Definizione**

**Architettura di von Neumann** (usata tipicamente dai **microprocessori**/PC): una **sola memoria** contiene sia i dati sia il programma. La **CPU** comunica con quest'unica memoria (la **RAM** del computer).
**Architettura Harvard** (usata tipicamente dai **microcontrollori**, inclusi i PIC): **due memorie separate** – una per i dati, una per il programma. La CPU può **leggere da entrambe contemporaneamente**.

**Esempio**

Con due memorie separate (**Harvard**) si può leggere il programma e i dati **nello stesso istante**, guadagnando velocità. Con una sola memoria (**von Neumann**) si deve scegliere: o si legge il programma, o si legge un dato, **mai entrambi insieme**.

**Attenzione**

L'**architettura Harvard** è più **veloce**, ma più **costosa** (**due memorie** invece di una). I PC usano von Neumann non perché sia la scelta migliore in assoluto, ma perché è **più economica** per un sistema general purpose. La **domanda tipica di verifica** su questo argomento: "cosa prevede un'architettura Harvard/von Neumann?" – rispondere sempre in termini di ***numero e ruolo delle memorie***, non di velocità (quella è una **conseguenza**, non la definizione).

**Nota**

Il nome "Harvard" viene dai calcolatori **Harvard Mark I-IV**, costruiti negli **anni '30-'40** all'università di Harvard sotto la guida dell'ingegnere **Howard Aiken**: a partire dal Mark III, questi calcolatori tenevano dati e istruzioni in **memorie fisicamente separate**. Il termine "architettura Harvard" in senso moderno, applicato ai microcontrollori, è stato coniato solo negli **anni '70**.

**Esempio**

Un PC può avere 24 gigabyte di RAM (circa 24 miliardi di byte), e un modulo RAM di quella capacità costa indicativamente un centinaio di euro: pochissimo per singolo byte. Un microcontrollore, invece, ha a disposizione soltanto un **centinaio di byte** di RAM (es. 100-130 byte) – ma quella memoria, integrata direttamente nella CPU con tecnologia Harvard, **costa molto di più** per byte rispetto alla RAM di un PC. Se la RAM di un PC costasse, byte per byte, quanto costa la memoria interna di un microcontrollore, un modulo da 24 GB arriverebbe a costare **centinaia di milioni di euro** – una cifra assurda per un PC. Per il microcontrollore, invece, va benissimo: gli serve **pochissima memoria** (poche centinaia di byte, non miliardi), perché deve svolgere un **compito limitato e ripetitivo**, non far girare un sistema operativo complesso con browser e applicazioni multiple.

# 3 Il circuito che genera il clock

**Definizione**

Il **clock** è il **segnale periodico** (un'onda che si alza e si abbassa continuamente) che scandisce l'avanzamento di un microcontrollore o microprocessore: ad ogni ciclo, la macchina esegue un **passo** del programma. Più veloce è il clock, più veloce esegue il programma la macchina – ma anche più calore produce.

## 3.1 Perché serve un circuito RC

Un **condensatore** si carica gradualmente, non istantaneamente – come riempire un secchio d'acqua attraverso un tubo: più stretto è il tubo (= più alta la resistenza), **più tempo** serve a riempirlo.

**Definizione**

Il **tempo caratteristico** di carica di un condensatore si chiama **costante di tempo** $\tau$ (tau):

$$\tau = R \cdot C$$

Dopo circa $5\tau$, il condensatore si considera **completamente carico** (oltre il 99% della tensione applicata).

**Esempio**

Con un **resistore** da 10 k$\Omega$ e un **condensatore** da 100 $\mu$F:

$$\tau = 10.000\,\Omega \times 0{,}0001\,\text{F} = 1\,\text{s}$$

Il condensatore risulta carico dopo circa $5\tau = 5$ secondi.

## 3.2 Da un condensatore che si carica a un'onda che oscilla

Collegando il condensatore a una **porta logica NOT** (invertitore) in un circuito a **retroazione**, si ottiene un comportamento **oscillante**:

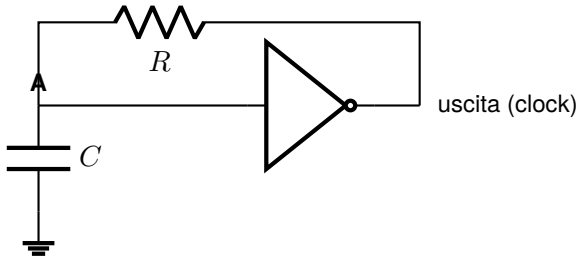



Figura 1: Oscillatore RC con porta NOT: il resistore $R$ collega l'uscita della porta di nuovo al nodo di ingresso A (retroazione), dove è collegato anche il condensatore $C$ verso massa. La tensione sul nodo A sale e scende ciclicamente, facendo commutare continuamente l'uscita della porta NOT: il risultato è il segnale di clock.

1. Il condensatore è scarico (0V): l'uscita della porta NOT è alta (es. 5V).

2. Questa tensione alta ricarica il condensatore attraverso il resistore.
3. Quando il condensatore raggiunge la soglia della porta NOT, l'uscita commuta a bassa (0V).
4. Il condensatore ora si scarica verso 0V attraverso il resistore.
5. Quando la tensione scende abbastanza, l'uscita torna alta, e il **ciclo ricomincia** da capo.

**Esempio**

Questo circuito, che oscilla in modo continuo e autonomo tra 0 e 1, è esattamente il segnale di **clock** che fa avanzare il microcontrollore. Si chiama **circuito RC** (resistore-condensatore).

## 3.3 Il problema: il resistore si scalda

**Attenzione**

Un resistore attraversato da corrente **si scalda**; scaldandosi, il suo valore di **resistenza cambia** leggermente. Questo altera la **velocità di carica/scarica** del condensatore, e quindi la **frequenza del clock** non resta **costante** nel tempo – un circuito RC puro **non è stabile**: la **velocità di esecuzione** del programma varierebbe con la temperatura, un comportamento imprevedibile e inaccettabile per un sistema affidabile.

## 3.4 La soluzione: il quarzo

**Definizione**

Un **quarzo** (cristallo di quarzo) è un componente che vibra a una frequenza estremamente **precisa e stabile** (**effetto piezoelettrico**), praticamente **indipendente** dalle piccole variazioni di temperatura. Inserito nel circuito di clock, **compensa** le derive del circuito RC, garantendo un'**oscillazione costante** nel tempo.

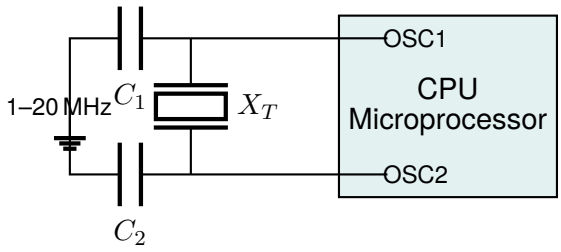



Figura 2: Circuito al quarzo: il cristallo $X_T$ è collegato tra i piedini OSC1 e OSC2 del microcontrollore, con due condensatori di carico $C_1$, $C_2$ (tipicamente 15–30 pF) verso massa – valori tarati per la stabilità e il tempo di avvio dell'oscillazione.

**Esempio**

Un microcontrollore economico può funzionare con un semplice **oscillatore RC** interno, arrivando al massimo a circa 20 MHz (**20 milioni** di impulsi al secondo) – una velocità **molto bassa** rispetto ai processori dei PC moderni, che viaggiano a diversi miliardi di impulsi al se-

condo (migliaia di volte più veloci). Ma cambiando resistore e condensatore, o passando a un **quarzo**, la **frequenza** del microcontrollore può essere regolata secondo le esigenze del progetto.

**Nota**

Esiste anche una **terza opzione**: il **clock esterno**, cioè un segnale di clock **già pronto**, generato da un altro dispositivo e fornito direttamente al microcontrollore (invece di farlo generare al PIC stesso con RC o quarzo). Si usa quando più chip nello stesso circuito devono lavorare ***sincronizzati sullo stesso clock***.

## 4 Cosa ricordare di questa lezione

- Un **microprocessore** fa solo calcoli, serve hardware esterno (RAM, memoria programma, I/O); un **microcontrollore** integra tutto in un solo chip.
- Sistemi **general purpose** (microprocessori, uso generico) vs sistemi **embedded** (microcontrollori, dedicati a un solo compito).
- Architettura **von Neumann**: una memoria unica per dati e programma. Architettura **Harvard**: due memorie separate, più veloce ma più costosa.
- Il **clock** è il segnale periodico che fa avanzare la macchina passo per passo.
- Un circuito **RC** genera un'oscillazione, ma non è stabile nel tempo (il resistore si scalda); un **quarzo** stabilizza la frequenza.

**Esempio**

**Da fare per la prossima lezione**: calcola la **costante di tempo** $\tau$ di un circuito RC con resistore da 4,7 k$\Omega$ e condensatore da 220 nF, e dopo quanto tempo il condensatore si può considerare **completamente carico**.